

AWS Certified Generative AI Developer — Professional

The Use Case

- Healthcare AI assistant platform — clinical decision support, medical documentation, patient engagement across hospitals
- Requirements:** multiple interaction patterns from varied hospital systems and devices; real-time assistance during time-sensitive clinical workflows; reliable operation during network disruption; model selection by medical specialty and clinical context; regulatory compliance while scaling

Task	Focus
2.3	Integrating with the enterprise — legacy systems, security, networking
2.4	Talking to the model at API level — timeouts, retries, streaming, routing

The notes are full of **specific numbers**. Treat them as guidance rather than hard limits — what matters is the **relative logic** (long timeouts for complex generation, short for simple inference), not memorising exact figures.

1. Flexible Model Interaction Systems

The first decision: synchronous or asynchronous

Pattern	Healthcare example
Synchronous	Clinical query — the doctor is standing with the patient. 4-5 second timeout.
Asynchronous (SQS)	Post-encounter documentation — the clinician has moved on to the next patient

Timeouts by model complexity

Complex generation -> up to 120 seconds
Simple inference -> 15-30 seconds
Clinical queries -> 4-5 seconds

Why clinical is so short: a doctor cannot wait. If the model hasn't answered in 5 seconds, **taking the fallback is better than waiting longer.**

Retry policy

Initial backoff: 100 ms
Backoff factor: 2 (100 -> 200 -> 400 -> 800)
Max retries: 3-5
Jitter: +/- 100 ms

Jitter prevents hundreds of clients retrying in lockstep and creating a second spike (as in Task 1.2).

Connection pooling

Pool size: 10-20 connections per instance
TTL: 60-300 seconds

Creating a new HTTPS connection per request is expensive (TLS handshake). **TTL exists because connections go stale** — load balancers change behind the scenes, so connections are refreshed periodically.

SQS configuration for async work

Visibility timeout: 5-15 minutes (10 minutes in the transcript)
DLQ after: 3-5 failed attempts

Visibility timeout must exceed expected processing duration. Set it too low and a second consumer picks up the same message — duplicate processing and duplicate model cost.

API Gateway request validators

max_tokens: <= 4096
confidence: >= 0.5-0.7
required fields: present

Transcript's healthcare version: required medical context fields, and terminology validated against standardised codes. **An invalid**

request never reaches the model — saving both cost and latency.

Error handling — retrieable vs non-retrieable

Code	Meaning	Action
429	Throttling	Retry with backoff
500	Server error	Retry
503	Service unavailable	Retry
400	Bad request	Do not retry — the request itself is wrong
401	Unauthorized	Do not retry — credentials are wrong
403	Forbidden	Do not retry — permission is missing

Rule: 4xx is your fault — resending the identical request is pointless. 5xx and 429 are service-side and may resolve on retry. Log and monitor each category separately.

Usage plans and burst

Steady rate:	10-50 requests/second
Burst:	2-3x the steady rate

Traffic is never uniform — the transcript notes spikes at **shift changes and morning rounds**, which burst capacity absorbs. Limits are set per department.

Client libraries

Cross-platform SDK wrappers with consistent error handling, retry logic, response parsing, and **timeouts calibrated to clinical workflow criticality**.

2. Real-Time Interaction Systems

The model takes 8 seconds. The clinician is watching a spinner. The answer must appear **as it is generated**.

Client-side buffer management

```
Buffer size: 5-20 chunks
Flush when:
  buffer is full
  100-500 ms have elapsed
  a semantic boundary is reached (sentence completes)
```

Semantic boundaries are the best trigger — half a sentence reads badly. Transcript's twist: buffers are configured so that **clinical information renders first**, with supporting detail following.

WebSockets

```
Ping frames: every 30-60 seconds
Idle timeout: ~10 minutes for interactive sessions
```

Pings keep the connection alive — network devices close idle connections. Use WebSockets when communication is genuinely **bidirectional**.

Server-Sent Events

```
Reconnection: exponential backoff from 1 second
Max delay: 30-60 seconds
Event IDs: to resume the stream after disconnection
```

Event IDs are the key feature. The connection drops at event #47; on reconnect the client sends `Last-Event-ID: 47` and the server continues from 48 — **the response is never regenerated from scratch**. The transcript uses SSE for patient education content precisely for this.

WebSocket vs SSE

	WebSocket	SSE
Direction	Bidirectional	Server → client only
Protocol	Its own	Plain HTTP
Complexity	Higher	Lower
Resume	Build it yourself	Built in via event IDs
Best for	Collaboration, chat	Streaming model responses

For LLM streaming, SSE is often sufficient — the client has nothing to send mid-stream.

Chunked transfer encoding

```
Chunk size: 1-4 KB
Preserve Transfer-Encoding headers in the
integration response template
```

The simplest option — plain HTTP streaming, for clients that can't speak a specialised protocol.

Mobile network handling

- Connection state detection
- Automatic switching between **Wi-Fi and cellular** — clinicians move through the building
- Buffering that adapts to available bandwidth and latency

UI feedback

```
Typing indicators, progressive rendering
Debounce: 300-500 ms
```

Debouncing batches updates instead of re-rendering on every token — balancing responsiveness against network and CPU cost.

Streaming error recovery

```
mid-stream failure
-> client-side recovery logic
-> attempt resume (event ID)
-> persistent failure?
-> FALL BACK to the full-response API
```

A complete non-streamed answer beats a broken stream.

3. Resilient FM Systems

SDK retry settings

```
Max attempts:      3-5
Initial backoff:   100 ms
Maximum backoff:  20 seconds    <- the cap matters
Jitter factor:    0.1-0.3
```

Without a maximum, exponential growth produces very long waits. The cap bounds it.

Multi-level throttling

```
Account level:  ~10,000 RPS
Stage level:   1,000-5,000 RPS
Route level:   50-500 RPS
```

Route-level limits are the important ones, because each model has different capacity. A complex-model endpoint might handle 50 RPS while a simple one handles 500. Account-level throttling alone would let the complex model be overwhelmed while still within the account limit. **Set limits from downstream capacity, not from preference.**

Fallback degradation paths

```
Generic:      primary model -> smaller specialised model
              -> cached responses -> static responses

Healthcare:   specialised medical model
              -> general model + medical prompts
              -> RAG over hospital knowledge bases
              -> rule-based system for critical functions
```

Note the third step in the healthcare chain: RAG is itself a fallback level. When the model degrades, answer from the hospital's own documents — less capable, but grounded and safe. Each level needs a quality threshold and transition logic.

X-Ray subsegments and annotations

```
Trace
+-- subsegment: preprocessing
+-- subsegment: model invocation
+-- subsegment: postprocessing

Annotations: model_name, input_complexity,
              output_quality_score, clinical workflow context
```

Annotations make traces filterable later — "show every slow cardiology request".

Multi-dimensional monitoring

Dimension	Why
Latency percentiles	p50, p90, p99 — p50 describes the typical user, p99 describes the worst experience . Averages hide the 1% waiting 30 seconds.
Error rates by category	429 vs 500 vs 400 mean different things
Token throughput	Capacity utilisation
Cost metrics	Spend per request

Alarm thresholds should derive from **SLAs**, not guesswork.

Circuit breaker tuning

```
Failure threshold:  50% over 10 requests
Recovery timeout:   30-60 seconds
Half-open traffic:  10-20% of normal
```

A small window detects failure quickly. Half-open sends only a fraction of traffic to verify recovery before fully reopening.

Multi-Region resilience

Model	Behaviour
Active-active	Both regions serve traffic
Active-passive	One serves, the other stands by

Health check interval: 30 seconds
Failover after: 3 consecutive failures (~90 seconds)

Route 53 routing policies must still **respect regional compliance** — the same constraint as Task 1.1, where failover could not cross a jurisdiction boundary.

Transcript addition: priority queuing so emergency requests bypass the normal queue — an ER request cannot sit behind routine documentation.

4. Intelligent Model Routing

- 1. Static config -> you decide, in configuration
- 2. Content-based -> decided by inspecting the input
- 3. Metric-based -> decided from performance data
- 4. Cascading -> escalate on low confidence

Static routing

Service	When
Parameter Store	Simple key-value configuration, low cost
AWS AppConfig	Feature flags, gradual rollout, validation, automatic rollback — as in Task 1.2

Either way, routing changes require **no code deployment**.

Content-based routing — Step Functions Choice states

```
Evaluated: token count, semantic complexity score,
           language detection, content classification

IF specialty == "cardiology" -> cardiology model
IF token_count > 4000        -> large-context model
ELSE                         -> general model
```

Transcript: patient data and **medical codes** drive the choice, with **parallel execution for multi-specialty consultations** — several specialist models running at once.

Metric-based routing

Store	Use for
DynamoDB	Current state, latest metrics, fast lookup
Amazon Timestream	Time-series data — trends and historical analysis over time

Transcript tracks **diagnostic accuracy, response quality and response time in Timestream** to continuously optimise model selection. The routing algorithm optimises for a stated objective — cost efficiency or response quality.

API Gateway mapping templates

```
Custom headers:  x-model-preference
Query parameters: ?complexity=high&quality=max
Payload analysis: inspect the body
plus a default when no selector is provided
```

Transcript applies **specialty-specific prompts and formatting** from clinical context headers — routing resolved before Lambda is even reached.

Model cascading

Confidence threshold: 0.7-0.9

New point: metadata passing between cascade levels. When escalating from the specialised model to the general one, pass along what was already attempted and what was found — otherwise the second model starts from nothing.

A/B testing

```
5-10% of traffic -> candidate model
90-95%           -> current model
```

- **Tagging and logging** so each request's model is known
- **Metrics collection** for both arms
- **Statistical significance testing** — a 2% improvement on 5% of traffic may simply be noise

Ensemble aggregation strategies

Strategy	Task type
Majority voting	Classification
Weighted averaging	Numeric predictions
Ranked fusion	Retrieval — merge several ranked lists into one final ranking (this is what Task 1.5's multi-store aggregation was doing)

Weights adjust dynamically based on historical performance.

Everything combined

```
request
-> API Gateway (routing hint from header/param)
-> AppConfig static configuration
-> Step Functions Choice (evaluate content)
-> Timestream performance metrics
-> model selected
-> cascade if confidence low (pass metadata)
-> 5-10% diverted for A/B
-> metrics written back to Timestream
```

Service limits and feature availability change. Verify specific numbers against current AWS documentation before the exam.